My senior project, Pi-Piper, set out to digitize how one of my local restaurants manages their back of house operations. For context, the restaurant I worked at and its two sibling restaurants are the businesses I was focused on, since I knew what had to be done and how the back of house functions. The back of the house mostly relies on paper for daily prep lists, weekly/monthly cleaning tasks, recipe books and otherwise. So I created an application using NEXT/Express web app intended to digitize all processes in the back of house for all three restaurants.

When I first had this idea, in the summer between my junior and senior year, I spent time in Bellingham. I landed a position as a grill cook at Chipotle. It was here where I was exposed to industry leading prep time series breakdown and analysis, and business insight integrated into every aspect of processes. They had each day mapped down to the minute from open to close. I found inspiration in the flexibility these programs offered their companies. It allowed them to plug in transfer management or employees with ease because of the standardized business practice ingrained into each location. This fueled my decision to create Pi Piper.

My backend mostly consisted of me finding the best ways to validate requests and running timely metric calculations. Express validator was my favorite to use across the entire backend of the application. Overall it took about an hour or two to apply to my API within the first week of creating my codebase. But since, I have not had a related issue and the return error messages are pretty and concise. If I could go back, I would not have picked express for my application stack. It is nice for simple backends meant to add another layer of serving data for some of the smaller projects I've worked on, but in what I have become interested in such as big data and data scraping in this project and beyond, express will not cut it down the road. My experience with express during this project has caused me to try and find an alternative that better suites the needs of my personal side endeavors. Such as Flask or fastApi, since I know data driven applications will always have the best, most capable libraries for AI training and data processing.

Split up into 4-5 major groups, the frontend is actually quite simple if you break it down. Styling was handled by the icon libraries, tailwind, and the MUI joy and material libraries. MUI was utilized heavily in the final half of development, because I brought it in to spruce up some of the more commonly used pages. The caching and data fetching libraries are likely the part of the application I spent the most time thinking about, and I enjoyed developing with. Tanstack is my favorite because it's easy to understand and can be used in just about any kind of application. Redux and Axios both worked well in their roles as well, but I found tanstack the most enjoyable and I want to try out some of their other parts of their project like tanstack forms or tanstack start. Form management overall wasn't all that complicated in this application. But, I did conclude that react-hook-form has been my favorite form management library. Now that I have made a few side projects as well as Pi-Piper now, it feels nice to find routine with my favorite libraries, which has made me far more comfortable and increased my productivity tremendously. Especially with react-hook-form I have been able to test out what works when it comes to form validation and pairing react hook form with other favorable libraries. like a react query, which leads to my next point…

TANSTACK QUERY

Tanstack is relatively lightweight, and I found the environment around tanstack query extremely strong. Very easy to find documentation and it seems like there is overwhelming support for them to continue to expand there expanding project.

React query felt like the right use case because there are a lot of potential for bad api requests which are required to complete to update the ui. allowing react query to handle the loading and errors saved me lots of headaches and helped improve the user experience two fold; since it enabled me to add adequate error messaging and spinners where necessary.

REDUX

Redux setup takes considerably longer and can be taken in multiple directions once implemented. It requires a strong understanding of the documentation and library itself to utilize it effectively and in the right places.

Became my option for accessing authentication across the app. an imperative requirement in this application, since it enables the app to be able to store three restaurants (and potentially more) operations/data. By accessing the stored restaurant id for the logged in user at load time, the user will always be served with the data specific to the restaurant they signed into.

AUTHENTICATION

As of now I have three levels of organization supported, owner, manager (admin), and employee (user). Owner provides business insight pages surrounding comparing metrics between the restaurant's as well as all other screens. Managers can access restaurant specific admin operation screens and prep operation screens. While employees only have access to prep screens.

I considered numerous different libraries which required lots of different proof of concepts. I ended up boiling my options down to next-auth(formerly auth-js), supabase, and clerk. Before upgrading my application to Next Js, this was a difficult decision. But the proof of concept with next-auth on an next application was seamless and fitting for the minimal amount of configuration that I need.

I considered eliminating the headache completely by leveraging a third party authentication service like clerk. But I ruled this option out eventually for a couple of reasons. The first being I want complete control over the way the organization is mapped out and assigned. In the future, the intricacies of the organization can be utilized to improve UX -- by allowing a simple attribute check in the middleware to easily tackle protecting routes for instance. Or it could be by

automatically filtering lists according to what the signed in user's role/department is, which could decrease the time spent filtering and searching. By allowing myself to add another layer of specificity in the user model, I don't limit myself in terms of assigning and modifying the numerous roles that make up a business like Republic Pi or its sister restaurants.

It's also important to consider the factor the user base plays in this application. Ultimately the number of users is finite since there are only three restaurant operations that it is tailored towards, which eliminates the need to prepare for quick scalability and data management. If this were more of a commercial project with the plans to deploy and expand as rapidly as possible, then clerk may become more of an option.

DATA WAREHOUSING

I designed my database to employ a data warehousing design technique. Although this added a layer of complexity, I learned a lot about the advantages of relying on warehousing concepts like dimension and fact tables rather than regular entities. In short, the warehousing design pattern requires that each table is either a "fact" or "dim" table. dimension or dim for short describe the business entities that help make up business processes (like dim_prep_item or dim_ingredient) and a fact table joins all the describing dim tables into bridge tables that describe the transactions (such as fact_daily_prep_list or fact_inventory) by storing observations or events like time of completion or quantity completed for example. When reading transactional data stored in the warehouse approach there are tremendous speed improvements as well. Since it reduces the amount of joins it takes to understand what a row represents. In a normalized transactional database, business events are fragmented across multiple tables requiring numerous complex joins that exponentially increase query processing time. Dimensional modeling prioritizes read performance by denormalizing data into fact and dimension tables, allowing most analytical queries to execute with just 1-4 joins instead of the 6-10 typically required in normalized systems. This design was best suited for storing the restaurant's daily transactions. The data store has been implemented with future growth planned for, since the performance will continue to improve with larger data sets.

GUI ELEMENTS

Custom Components

Custom components are an easy way to bind UI state closely with schema's. Probably the reason that I fell in love with react is how you can create focused components that are flexible to your liking. For example, with my use of the data warehousing database design, creating pages that update transactional data can be a challenge due to the complexity of the tables. But, custom components make it easy to mutate existing entities into its underlying business transactional entity that it represents. POST'ing or PUT'ing data objects directly from custom components is a perfect jumping off point for any junior developer learning the ropes of frontend development. By attempting to develop frontend state models that map as closely to the data

store as possible, it creates a simplified structure that tries to avoid data translation. The idea of custom frontend components was the most attractive aspect of React and Next when I initially chose to learn them. Even today, the amount of inventiveness you are allowed while still being able to create a highly logical layout impresses me. It is no surprise to me why there is such a surge towards Next JS. It's very inviting to beginners while still hitting high performance and DX marks.

Material UI

There were multiple parts of MUI that I included across the application, like MUI joy or MUI material. I wasn't able to completely utilize material components, a lot of the custom components and styling was custom css/tailwind, but core functionality usually include a MUI component. Plus, their material styling library helped create a fitting theme that resembles the tone of the application. There is little excess in this application and it is mostly focused on simulating business processes that it felt right to ensure the process screens were not too fancy and overdone, material fit that bill. Where there is more insight, notifications, dashboards, this is where Joy fits in. Joy was more applicable here because it helps create metrics that pop off the screen and are clear. Overall I think material was a good pick as an excess styling library that added a little extra enhancement to the components that see a lot of interaction.

Tailwind/Next Install

Next install configurations are extremely opinionated but extremely effective. Although it really doesn't feel like I need to switch styling options, all together my knowledge surrounding web design is limited relative to the rest of my tech stack. Before beginning my journey learning NEXT I had never used tailwind in any of my apps. I felt like it instantly enhances the feel and has as good or possibly better syntax to its competitors that I had a little bit of experience with, mostly bootstrap. From what I understand about the current state of web development, tailwind has a chokehold on the industry due to how deep NEXT has entrenched tailwind setup into a project setup. But, I would be interested to see if it's even worth it to mainly depend on any other styling library effectively today.

OVERCOMING CHALLENGES

Provider Pricing Scraping

My contact with the restaurant has allowed me to contact some of the local providers in an effort to try and get authorization to do site scraping. Because you are only able to see pricing based on your subscription, any scraping I would do would be under their authorization, therefore putting their account at risk of disrupting service if the scraping were to ever be flagged at some point. The last thing I wanted to do was impede on any standing relationships or business that the restaurants have going, so this was a major hurdle and I am talking as much precaution as I can, while keeping the stakeholders in the loop about my progress for complete transparency. I have invested plenty of time into finding the correct solution to handle

the scraping and data store updating, and I have a solution standing by ready to try with the right authorization and login credentials from the restaurants if it gets to that. But, for now, the screen is filled with mock data in an effort to exemplify the benefits and how the tool can be implemented into business practice.

Typing(Drizzle ORM)

Stock Next configurations have led me to only use typescript when developing. Before the design process it was a major concern over how to model UI state and endpoint responses across an application of this size. By no means a large app, but the different types in each screen can build up and create confusion quickly. Drizzle was a late introduction to the tech stack that focused on this issue. It took a lot of busy work to implement, but as soon as I figured out the right way to utilize the same types in as many components and files as possible, it proved to be very worth it. Drizzle or another ORM alternative like Prisma have found themselves a solid place in my personal tech stack of choice moving forward. Through my own personal side projects and proof of concepts, I've found ORM's extremely beneficial on a few fronts. The first being the piece of mind with typing in what is otherwise a very delicate system. The type safety starts from the second the database is queried, since the types are derived from the database schema. Admittedly, the process for me to introduce drizzle into my application was quite the challenge. Pi-Piper is a data driven application whose data store contains lots of transactional tables, which are made up of plenty of foreign references. This made for the labor intensive task of manually creating all connections within the application schema file. Also, the task of making developments locally and managing the migrations between changes in the database for each attribute change caused some frustration. Looking back on it, after going through the initial learning curve of utilizing an ORM, the benefits still seem just as important even after my initial developer experience. Although there were a few painstaking hours in the beginning, the overall type safety and general maintainability of my application should remain fairly high for the scope of this application.

NEXT, NEXT, NEXT

Upgrading to next so late into the planning phase may have thrown off my schedule for development a little. But it launched me into the best thing I have put my hands on as a developer. In hindsight I was lucky to have found Next when I did, because when I began to start to learn React last summer, I barely knew about Next. I was able to put the time in with React concepts like the role that hooks play or the importance of modularization and the separation of concerns. My time over the summer and up until deciding to upgrade to Next allowed me to establish a strong fundamental understanding of the uses and best practices of the framework Next is built on. Which overall made for a smooth transition for me once it was all said and done. Without Next, a lot of peoples main concern over React, routing and page management, would have likely been a hard challenge to manage. But Next almost made that issue a no brainer immediately with the app router. On top of that, Next does a good job of improving upon some of the vanilla react library adaptors. Like next-auth which basically handled most of the authentication logic, or next-pwa which allows me to utilize the Progressive

Web App service worker technology to improve page load times and allow the app to still work when offline using the service workers page caching system. Next's helpful integration with these technologies was another key contributor to helping move the project along.

WHAT MAKES PI PIPER WORTH IT?

Future Facing

Adding a tool to help document and assist analyzing inventory and prep trends in the intermediary process of the ingredient's lifecycle can help fill in the gaps to identify inefficiencies. In between point of sale of menu items and point of sale of ingredient items, this system can also be integrated to help track some of the more granular inventory transactions so that there is more robust inventory management. Such as prep employees submitting to the app when waste has happened (whether something is not selling and has to be thrown out, or has been spoiled for some reason). This way, the system allows management to make targeted operational decisions based on whether waste occurs during the initial prep stage or during the assembly and sale process.

Having complete control of the kind of information that Pi Piper would be logging is a tremendous business advantage. Whether the company decides to leverage this with Pi Piper or with another service is up to them. But the point is that there are plenty of tools that are built on data like this that can transform the way they do business. The data is catered to work with these systems so that in the future they can implement some of the following insight and automation ideas i've had during the planning and development phases of this project:

- Time series analysis and planning of daily prep lists
- Inventory Stock level analysis
- Stock shelf life cycles

Immediately following this iteration of this code release I plan to implement a photo upload service linked with some optical character recognition and LLM technology to update inventory based upon order invoices.

If the owner ever decided to sell their three restaurants, having a comprehensive picture of each restaurant's back-of-house operations provides a significant competitive advantage. Pi-Piper delivers visibility by giving stakeholders clear insights into trends and stock levels, empowering them to make informed labor and menu decisions. In my opinion, this is reason enough to integrate Pi-Piper. The power of data driven applications allow you the power to transform operational data into tangible business value, as potential buyers can see concrete evidence of efficiency, growth patterns, and operational stability, beyond what is already in place.

Housing the operations of all three restaurants under one domain has numerous advantages. First, it begins to standardize restaurant practices allowing them all to be managed under one roof and compared to one another. Next, allowing managers and owners to compare restaurant locations that foster collaboration and a better tool to compare the restaurants, comparison can be helpful for incentivizing by creating number-based goals.

Most importantly, Pi-Piper allows remote management. With implementation and refinement of the application in use, it could be used to virtually set prep lists, set task lists, or create carts that should be ordered by those in charge. This creates the opportunity to cut labor if possible and focus more so on the core of the business. Plus, adding a specialized application at all three restaurants opens up the field of those who would be able to manage, since all that must be done is neatly documented. This is a major aspect and a useful tool to mitigate staffing and transfer concerns.

Pi-Piper represents not just a digital transformation of restaurant operations, but a strategic investment in the future of these businesses. By centralizing data management across three restaurants, standardizing processes, and creating powerful analytical capabilities, the application provides immediate operational efficiencies while establishing a foundation for future features. The system has been designed and developed with future outlook heavily in mind, and I believe ultimately contributes to the restaurant's long-term value and sustainability.